

# Synthesis with CppHDL

CppHDL has an experimental synthesis backend that converts C++ RTL directly to technology-independent gate-level Verilog. It uses the same operation graph as native graph simulation, with separate hardware mapping and retiming passes. It does not generate SystemVerilog and feed it into another synthesis tool: neither graph export nor gate mapping requires Yosys.

The result is a digital implementation that can be tested with Verilator and passed to a technology-specific flow. It is **not** an FPGA bitstream or an ASIC standard-cell netlist. Generic gate counts are not FPGA LUT counts, and the built-in delay estimates do not establish physical timing closure.

## Quick Start

Build the converter using the repository's build instructions. Synthesis additionally needs Python 3 and a C++17 compiler to build its generated graph emitter. Verilator is needed for gate-level regression tests, not for generating the netlist.

Run commands from the repository root. This example synthesizes the integer arithmetic and register design in tests/math.cpp:

```
1 cmake --build build --target cpphdl -j2
2 build/cpphdl --synth --top cpphdl_top --module SynthMath \
3   --output build/synth-doc-math synth/tests/math.cpp
```

`--top` selects the C++ root object or module class. In this example, `cpphdl_top` is an instance of `SynthMath`. `--module` selects the name of the emitted Verilog module; it does not select a C++ class.

The output directory must be **new or empty**. Choose a different directory for a second run; the driver does not overwrite a previous result. On success, it prints the path to `gates.v`. Check `manifest.json` for `"status": "complete"`.

Design preprocessing options go after `--`, for example:

```
1 build/cpphdl --synth --top MyTop --module MyTop \
2   --cxx clang++ --output build/mytop-synth design.cpp \
3   -- -I/path/to/design/include -DWIDTH=32
```

The driver supplies `-std=c++17`, `-DSYNTHESIS` and the repository include path to the graph frontend. Keep testbench-only code behind an appropriate guard. Options after `--` configure design parsing, not the later graph-emitter build. `--cxx` selects that emitter's compiler; its default is `$CXX`, or `c++` if unset.

## How It Is Implemented

The synthesis pipeline has five steps:

```
1 C++ Module hierarchy and port bindings
2   |
3   Clang AST graph lowering
4   |
5   Typed operations, state and memory
6   |
7   Optional timing-driven retiming
8   |
9   Generic bit-level gate mapping
10  |
11  gates.v + reports
```

1. **Resolve the design from the AST.** `CppGraph.h` resolves the selected module hierarchy, its `_assign()` bindings, helper calls, work methods and strobes. Supported loops and calls are expanded into graph operations. Register next-state expressions and commits become explicit state boundaries; memory accesses retain their enables, ordering and clock ownership.
2. **Build the shared representation.** `include/cpphdl_graph.h` represents values as bit references and typed operations. It records ports, register transitions, memory operations, clocks and instance scopes. Connections and field selection can be wiring rather than copied temporary values. Shared passes fold constants, simplify expressions and find live dependencies.
3. **Apply optional retiming.** `synth/timing.cpp` estimates combinational and register-boundary delays. `synth/retiming.cpp` moves existing boundaries or inserts pipeline registers according to an explicit rule. Without a CLI rule or source annotation, this step reports timing without retiming the design.
4. **Map operations to generic gates.** `synth/Mapping.cpp` expands arithmetic, comparisons, shifts and selections into bit-level logic. Identical gates can be shared. Registers preserve their clock, edge and reset information. Memories are expanded into register storage with read muxes and decoded writes.
5. **Write Verilog and reports.** `synth/Verilog.cpp` emits the operation netlist and mapped gates. The driver checks that the mapping report contains only generic cells or explicitly preserved boxes before reporting completion.

`cpphdl --synth` dispatches to `synthesize.py`. That driver invokes the internal `--lower-synthesis-graph` entry point to produce `graph.cc`, then compiles it together with `Main.cpp` and the synthesis backend. Running this emitter performs retiming, mapping and file generation. The host C++ compiler builds the graph program; it does not determine the circuit by lowering the design through LLVM machine-code IR.

## Source Map

The main implementation files are:

- CppGraph.h: C++ AST to graph, lifecycle and clock ownership.
- cpphdl\_graph.h: shared operations, state, memory, simplification and graph serialization.
- ScheduledGraph.cpp: scheduled HLS blocks and control flow to graph operations, registers and memory ports.
- StreamPipeline.h: elastic HLS pipeline construction, operand alignment, valid propagation and feedback state.
- Mapping.cpp: generic bit-gate mapping and cell reports.
- Verilog.cpp: operation/gate Verilog emission.
- timing.cpp, timing.h: delay model and path estimates.
- retiming.cpp, retiming.h: retiming rules and passes.
- KeepBoxes.h: explicitly preserved operation-level regions.
- synthesize.py, Main.cpp: command-line orchestration and the generated graph's synthesis entry point.

Native simulation and synthesis share design semantics, not optimization goals. CPU execution scheduling belongs to the native backend; hardware pipeline placement belongs to synthesis. See shared lowering.

## Generated Files

Each run keeps its intermediates and diagnostics in `--output`:

| File                          | Purpose                                                                |
|-------------------------------|------------------------------------------------------------------------|
| <code>gates.v</code>          | Mapped generic gate-level design.                                      |
| <code>operations.v</code>     | Operation-level design after any requested retiming.                   |
| <code>gates.json</code>       | Mapped ports, cells and connections.                                   |
| <code>timing.json</code>      | Estimated timing, retiming results, keep boxes and HLS regions.        |
| <code>blackboxes.v</code>     | Empty declarations for explicitly external implementations, when used. |
| <code>manifest.json</code>    | Run status, commands, clock arguments, cell counts and timing.         |
| <code>graph.cc</code>         | Original graph-construction program with <code>makeGraph()</code> .    |
| <code>retimed_graph.cc</code> | Graph after processing, including retiming when enabled.               |
| <code>emit</code>             | Compiled graph-emitter executable.                                     |

| File                             | Purpose                                                       |
|----------------------------------|---------------------------------------------------------------|
| <code>lower.log</code>           | AST lowering diagnostics.                                     |
| <code>compile-emitter.log</code> | Host compiler diagnostics.                                    |
| <code>emit.log</code>            | Graph validation, retiming, mapping and emission diagnostics. |

Preserved boxes have operation-level implementations included in the emitted Verilog and also written to `keep_boxes.v` for inspection. Compile `gates.v` for gate simulation, not all `.v` files together: `operations.v` is an alternative representation of the same design, and the separate box file duplicates definitions already included in the netlist. Preserved boxes are deliberate exceptions to generic gate expansion, not a claim that a DSP or other technology primitive has already been selected.

## External Math Blackboxes

Use `CPPHDL_BLACKBOX=<module>:<delay_ns>` on a pure free/static function when its implementation must remain outside synthesis:

```

1 [[clang::annotate("CPPHDL_BLACKBOX=my_inverse_sqrt:0")]]
2 int64_t inverse_sqrt(int64_t value); // Or a body for native simulation.

```

Unlike `CPPHDL_KEEP_BOX`, this annotation prevents lowering the function body. It works in the synthesis AST frontend and in `ClockedPipeline` calls, including ordinary HLS RTL conversion. A declaration without a body is sufficient for synthesis. Native C++ execution still requires a definition. `ClockedDelay` calls, reference/pointer arguments, non-static methods, and floating-point or greater-than-64-bit arguments/results are currently rejected.

The external module must provide `INPUT_BITS` and `OUTPUT_BITS` parameters, an input `args[INPUT_BITS-1:0]`, and an output `result[OUTPUT_BITS-1:0]`. Each integer argument occupies 64 bits (argument zero is least significant), sign-extended or zero-extended from its declared C++ type. Results use their declared integer width. Functions must have no externally visible side effects.

Instances carry `keep` and `keep_hierarchy` attributes. Mapping preserves the call, even with constant arguments, rather than evaluating its body or replacing it with zero. `blackboxes.v` contains **empty declarations**, not functional models. Provide your own implementations when linking or simulating the design; do not compile these declarations alongside those implementations.

The annotation specifies combinational delay in nanoseconds, not cycle latency. Zero is allowed as an explicit abstraction. All such boxes have zero cycle latency; registered implementations require a separate handshake/latency adapter. `timing.json` lists `external_blackboxes`, while `manifest.json` lists `external_implementations_required`. Gate counts exclude their unknown internal cost. A timing result with zero-delay boxes

does not establish physical timing. See the LLM arithmetic example for real add/mul logic surrounding external nonlinear functions and test-only Verilator models.

On failure, read the log named by the error and `manifest.json`. Intermediate files may exist even when the run failed; their presence alone does not mean synthesis succeeded. `--tool-timeout SECONDS` changes the execution timeout per external invocation (default 300 seconds), not the requested circuit timing.

## RTL, Clocks and Reset

For an ordinary single-clock module, one rising `clk` edge implements one `_work(reset) / _strobe()` transaction. Bind ports in `_assign()`, calculate next register values in `work`, and commit registers and buffered memory writes in `strobe`. Synthesis does not invent missing child calls or memory `apply()`. Without retiming, it does not add pipeline latency.

The ordinary graph frontend removes top-port direction suffixes: `a_in` becomes `a`, and `sum_out` becomes `sum`. Its lifecycle reset input is `work_reset`. For a hierarchy containing scheduled HLS wrappers, top-port suffixes are kept and reset is named `reset`. Inspect the emitted module declaration when writing a testbench; do not assume it has the ordinary SV converter's exact port names.

To synthesize named clocks, declare them explicitly:

```
1 build/cpphdl --synth --top cpphdl_top --module MainAndSecondary \  
2 --primary_clock main_clk 120000000 \  
3 --secondary_clock secondary_clk 40000000 \  
4 --output build/synth-two-clocks \  
5 synth/tests/multiclock/main_and_secondary.cpp
```

For independent clocks, use repeated `--clock NAME HZ` options instead. Do not mix the two forms. Frequencies are positive integer Hz; a secondary clock requires a primary clock whose frequency is at least as high. These declarations create clock inputs and ownership metadata, not clock dividers or synchronizers.

Named clocks use `_work_NAME(bool reset)` and `_strobe_NAME()`; optional falling-edge processes use `_work_neg_NAME` and `_strobe_neg_NAME`. Each register must have one matching work/strobe clock and edge. Separate processes and mapped flops retain that ownership. Multiple owners and cross-clock `_next` reads are rejected.

In the ordinary named-clock flow, `_reset_pos_NAME()` and `_reset_neg_NAME()` provide active-high asynchronous reset through `work_reset`. Here `pos/neg` selects the clock edge, not reset polarity. Each handler must supply an unconditional constant reset value for every register in that clock/edge. It cannot access or clear memory. Domains without a handler retain clocked reset behavior. See clock and reset details and the multiclock tests.

## Memory

The graph supports a memory written by one clock/edge and read by other domains. Call `apply()` from the writer's strobe. Ordinary reads see committed storage; `pending()` forwards earlier buffered writes within the writing domain only. Registered readers at coincident edges sample the old value. This digital model does not make unsynchronized same-address collisions safe in physical RAM.

`operations.v` retains memory arrays. **The current generic mapper expands them into flip-flops and muxes; it does not infer block RAM or SRAM macros.** Large arrays can therefore produce large gate netlists. Memory contents have no invented initialization or reset. Write locations before reading them, unless the design supplies another explicit initialization mechanism.

The generic mapper rejects memories larger than 16,777,216 bits or 1,048,576 rows. Those are implementation limits, not recommended sizes. Technology memory mapping is separate work; selecting an HLS memory policy does not turn generic gate mapping into a BRAM mapper.

## HLS Integration

No `--hls` flag is needed. The converter detects scheduled wrappers in the selected module hierarchy, including children, and selects the scheduler from the wrapper type:

- `ClockedDelayer<T, ...>` schedules a method as an ordered multicycle operation. Loops and memory accesses can introduce clock boundaries. Its native C++ execution is a functional reference, not necessarily the generated FSM's cycle schedule.
- `ClockedPipeline<T, STAGES>` constructs a pipeline that can admit a call every clock when downstream is ready. Synthesis can add stages to meet its estimated timing target while preserving `II=1`.

Ordinary conversion emits the scheduled RTL. `--synth` additionally exports the actual schedule directly into the shared graph; it does not reparse emitted SV or synthesize the native wrapper's reference `_work()` implementation.

Pipeline feedback is intentionally latency-dependent. Overlapping calls can read the same old committed object state; adding stages can change the result of an FSM that assumes immediate feedback. There is no automatic stale-state interlock. Use a latency-tolerant algorithm, an explicitly ordered RTL recurrence, or `ClockedDelayer` when calls must observe each preceding update.

See HLS contracts and the streaming HFT example. The HFT example combines three HLS pipelines with manually registered framing, validation and checksum logic; timing analysis also checks the logic outside the HLS regions.

## Retiming

Request retiming explicitly, for example on the feed-forward regression:

```
1 build/cpphdl --synth --top cpphdl_top --module SynthRetiming \  
2   --retiming fit_pipeline_retiming --clock-period-ns 2.5 \  
3   --output build/synth-retimed synth/tests/retiming/logic_retiming.cpp
```

`keep_behaviour_retiming` moves existing register boundaries without changing cycle behavior. Its bounded search may fail to meet the target; inspect `target_met` even if the run succeeds. `fit_pipeline_retiming` adds latency and fails if the transformed design still cannot meet its estimated target. Ordinary feedback and delayed-HLS retiming can require a ready/commit transaction protocol and reduce throughput. Streaming HLS retains its ready/valid interface and `II=1` instead. These are different contracts, not interchangeable wrappers.

`--retime-module INSTANCE_PATH` restricts a CLI rule to an instance and its descendants. Source annotations can also select regions or preserve a box with a declared delay. `--delay-scale FACTOR` scales the built-in estimates; it is not calibration against a cell library. Declared clock frequencies alone do not enable retiming: supply a period or source rule.

Read `retiming.md` for annotations, one-clock functions, preserved boxes, feedback protocols, memory restrictions and latency interpretation. Tests must account for the transformed contract, not only compare outputs on the source model's original clock numbers.

## Tests and Current Limits

Enable the synthesis regressions in an already configured build:

```
1 cmake -S . -B build -DCPPHDL_BUILD_TESTS=ON -DCPPHDL_BUILD_SYNTH_TESTS=ON  
2 cmake --build build --target cpphdl synth_graph_checks synth_mapping_checks \  
3   synth_schedule_checks synth_retiming_checks synth_stream_checks -j2  
4 ctest --test-dir build -R '^synth_' --output-on-failure
```

Set `CPPHDL_SYNTH_VERILATOR` to the Verilator executable at configuration time if it is not on `PATH`. Without Verilator, CMake registers graph-level checks but not gate-simulation regressions. To run only the arithmetic example:

```
1 ctest --test-dir build -R '^synth_math$' --output-on-failure
```

The suite covers integer math, parent/child pipelines, independent clocks, positive/negative edges, memory ownership, asynchronous reset, retiming, kept boxes and scheduled HLS. Tests

compare native C++, graph execution and/or Verilated gates as appropriate. The HFT regression additionally checks packets, backpressure, reset, error metadata and continuous-word throughput after retiming.

Important boundaries of the current implementation:

- The AST frontend accepts a defined subset of C++, not arbitrary software. Host callbacks and unsupported event contracts are rejected. It does not synthesize floating-point library calls merely because they compile in C++.
- Graph operation nodes currently use at most 64-bit words. Wider wiring and bitwise values can span nodes; general arbitrary-width arithmetic support is not implied by that representation.
- Ordinary module hierarchy is lowered into the graph with instance scopes; it is not automatically retained as a hierarchy of gate-level modules. Explicit keep boxes provide selected preserved boundaries.
- Scheduled HLS export and streaming retiming have narrower clock, memory and reset support than ordinary RTL synthesis. Consult the HLS and retiming contracts before combining those features.
- The backend does not insert CDC synchronizers or prove crossings safe, and does not model metastability, reset recovery/removal or physical RAM collisions.
- Delay estimates omit target-library, placement, routing, fanout and clock-skew effects. There is no automatic FPGA LUT/DSP/BRAM or ASIC cell selection. Gate simulation checks digital behavior; final acceptance requires the intended technology's synthesis and timing checks.

Use `build/cpphdl --synth --help` for the complete command-line option list.